Test Report — Who added each card (`added_by_user_id`)

PR: kuantorflow#169 (+ kuantorflow_automation#31) · **Issue:** kuantorflow#89 · **Date:** 2026-08-02

Summary

`flashcards` gained an `added_by_user_id` column recording the signed-in user who saved each card, `NULL` for anonymous visitors. Verified against the real local MySQL database through the actual save route, including that a forged id in the POST is ignored and that the foreign key behaves as #165 requires. A second defect was found and fixed in the same PR: sessions created before #148 looked signed in but carried no user id, so their cards saved unattributed.

What changed

- `schema.sql`: `added_by_user_id INT NULL AFTER topic`, `idx_added_by`, and `fk_flashcards_user` with **`ON DELETE RESTRICT`** (settled in #165 — account deletion resolves the cards first, so a forgotten step must fail loudly). `ALTER` statements added as comments for existing databases, following the `pos` convention.
- `utils.save_flashcard(entry, added_by_user_id=None)`: the owner is a **separate argument**, deliberately not a member of `FLASHCARD_FIELDS` — `entry` is built from form data, so a key of that name in it must never become an owner.
- `app._current_user_id()`: the single reader, resolving from the session only. Never from request data — the review popup posts hidden fields.
- `app._save_and_log()` passes it, which covers all four save paths (review popup, *Add All*, automatic add on lookup, Mykola's chat saver) in one place.
- `app.drop_identity_from_before_the_users_table()`: a `before_request` (registered ahead of `require_keyword`, so gated requests are repaired too) that drops an identity carrying no `id` key. An `id` of `None` is left alone — that is a sign-in whose users row could not be written, tolerated by design.

Automated tests

- **Added:** `tests/test_card_ownership.py` — 22 tests. The column reaches the `INSERT` (fake connection); every save path records the owner, signed-in and anonymous; a forged `added_by_user_id` is ignored three ways; a sign-in with no users row saves anonymously; the field is absent from the flashcards page, the deck page and the review popup; and the pre-#148 session repair, including that the gate pass survives it.
- **Updated:** `conftest.py` — the `saved` fixture captures the new argument in `owner_ids`; `user_client`'s session carries an `id`. Three tests stubbing `save_flashcard` directly accept the new keyword.
- **Suite result:** `pytest -m "not live"` — **227 passed** (was 205), 1 skipped, 6 deselected.

- **Negative check:** with the app change reverted, 10 of the 22 fail. The remainder are regression guards for behaviour that must not change.

Manual & browser verification

Driven against the **real local MySQL database** (not stubs), through the actual `/cards/add` route, with all probe data removed afterwards.

- Applied the column, index and FK locally → `SHOW CREATE TABLE flashcards` confirms `added_by_user_id int DEFAULT NULL`, `KEY idx_added_by`, and `CONSTRAINT fk_flashcards_user … ON DELETE RESTRICT`.
- Anonymous save through `/cards/add` → stored `NULL`.
- Signed-in save → stored that user's id.
- Save with `added_by_user_id=<another id>` in the POST → stored `NULL`; the posted value never reached the card dict.
- `INSERT` naming a nonexistent user id → rejected, `1452 … foreign key constraint fails`.
- `DELETE` of a user whose cards were still attached → refused, `1451 … Cannot delete or update a parent row`. This is the `RESTRICT` guarantee #165 depends on.
- Repair loop end to end, on a database that had never held a `users` row: a session in the pre-#148 shape was dropped on the first request → `upsert_user` wrote a real row → a card saved afterwards carried that id.
- Cleanup: all probe cards and probe users deleted; `users` returned to 0 rows.
- Post-merge confirmation on real data: card `digital` (id 85) carries `added_by_user_id = 7`, the author's real users row, while `literally` (saved earlier from the stale session) remains `NULL`.

Result

Pass. Every acceptance criterion in #89 met, and the `ON DELETE RESTRICT` decision from #165 verified rather than assumed.

Flagged for follow-up:

- **Deployment gap (hit in production).** `schema.sql` uses `CREATE TABLE IF NOT EXISTS`, so re-applying it does **not** add a column to an existing table, and #89's `ALTER`s live in that file only as comments. The deployed code inserted the column before it existed and every save returned 500 until the three `ALTER`s were run by hand on PythonAnywhere. Every future column has the same trap; an idempotent apply script would remove it.
- The 50 pre-existing local cards (and their server equivalents) have no owner and cannot be backfilled — nothing recorded who saved them. Under #162 they become admin-only, which is the safe default but a one-way door.
- `NULL` breaks intuitive comparisons: #127's "everyone else's cards" needs `IS NOT NULL` / `<=>`, never `!=`.